

• Topologically organised n/w's —

Principle — No. of biological neurons come together & organise themselves to perform some specific task without any definite goal for every neuron.

Most commonly used topologically organised n/w model.

- Kohonen's model — It is also referred as,

self organizing map / topologically preserving map /

self organizing feature map.

SOM's —→

① Unsupervised learning → Learning w/o teacher.

- Desired o/p response will not be given.
- These type of n/w attempts to discover some special features & patterns from available data w/o using external help.
- one of technique to solve some prob →

① Clustering → Here given data will be partitioned into disjoint subsets such that most similar patterns will take same cluster & non similar patterns take different clusters.

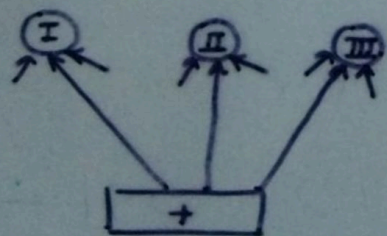
→ How to assign o/p pattern to a particular cluster?

⇒ Evaluate avg squared distance betn o/p pattern & cluster centroid.

⇒ Types of n/w's for clustering

- ① Winner Take All n/w
 - (a) Hamming net
 - (b) max net
 - (c) min HD classifier
 - (d) simple competitive learning.
 - (e) K-means clustering.
- ② Topologically organized n/w.

① Winner Take All N/w → connection wts = cluster centroids



1. Calculate distance betn o/p pattern & cluster centroids.
2. For whichever o/p node distance is min. that is called winner node
3. Then only winner node wts are updated.

- Two ways of finding winner node →

- a. Calculate squared Euclidean distance betn o/p vector & all stored vectors. for which stored vector distance is min is called winner node.
- b. Calculate Hamming distance betn o/p vector & all stored vectors. for whichever stored vector dis. is min is called winner node.

• To calculate distance betn stored vectors & ip vectors use $\rightarrow$ (2)

a) Hamming n/w $\rightarrow$ Tells about ip vector closest to which stored vector

- It calculates Hamming distance betn stored vector & ip pattern.

- Hamming distance = no. of differing bits.

- The n/w consist of input & output layer

- If P class Hamming n/w is there, no of neurons in o/p layer = P.

- wts on links betn two layers represent components of stored patterns.

- $-H(i, i_3)$ $-H(i, i_2)$ $-H(i, i_1)$

T ω_{31} ω_{32} ω_{33} P neurons

Let $i_p: P = 1 \dots P$

= set of binary stored vector with dimension n.

$\therefore i_1 = \text{centroid for 1st neuron}$

$= \{\omega_{11} \omega_{12} \omega_{13}\}$

$i_2 = \{\omega_{21} \omega_{22} \omega_{23}\}$, $i_3 = \{\omega_{31} \omega_{32} \omega_{33}\}$

$-H(i, i_3) = -ve$ of Hamming dis betn stored vector i_3 & input vector i .

- Connection wt from j th ip to p th o/p node is given by $\omega_{pj} = \frac{i_p j}{2}$

$\therefore$ wt vector is given by,

$$W = \frac{1}{2} \begin{bmatrix} i_1^T \\ \vdots \\ i_p^T \end{bmatrix} = \begin{bmatrix} \omega_{11} & \omega_{12} & \dots & \omega_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ \omega_{p1} & \dots & \dots & \omega_{pn} \end{bmatrix}$$

- Each upper level node is associated with threshold $\Theta = -\frac{\text{no. of no}}{2}$

$$\text{i.e. } \Theta = \begin{bmatrix} -n/2 \\ \vdots \\ -n/2 \end{bmatrix}$$

- For new vector i presented to n/w, o/p can be given as

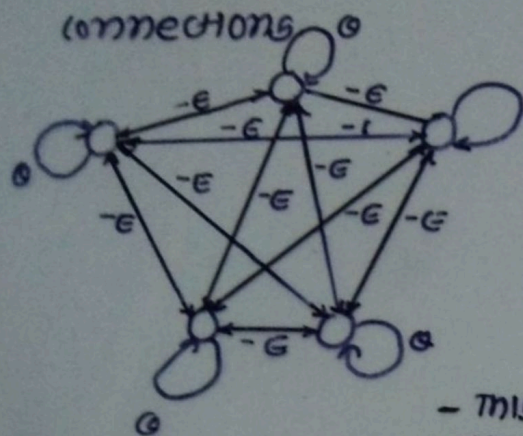
$$O = Wi + \Theta = \frac{1}{2} \begin{bmatrix} i_1^T i \\ \vdots \\ i_p^T i \end{bmatrix} - \begin{bmatrix} n/2 \\ \vdots \\ n/2 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} i_1 \cdot i - n \\ \vdots \\ i_p \cdot i - n \end{bmatrix} = \begin{bmatrix} -4 \\ -3 \\ -2 \end{bmatrix} \text{ eg}$$

is used for purpose of scaling.

Hamming dis are $\rightarrow 4, 3, 2$

$\therefore$ node with min Hamming dis. is called winner node.

- Maxnet → Tells about which stored pattern is closer to unknown input pattern.
- It is one layer recurrent n/w involving both excitatory & inhibitory connections.



Excitatory connections = self fib loop with weighing $w_{ee} = 1$
 $= \theta [+ve]$

Inhibitory connection = from each node to every other node in n/w
 $= \epsilon [-ve]$.

- This n/w determines which node has initial highest value.

- It is an iterative procedure.
- Here wts can be chosen in such a way that single node whose value is initially max prevails & activation of other nodes subsides to zero. $\uparrow$ is called winner node.

- Normally self excitation wt $\theta = 1$

- mutual inhibition w_{ee} , lateral interaction $w_{ee} \leq \frac{1}{\text{no. of nodes}}$

- net o/p is calculated as,

$$\text{net} = \sum_{i=1}^n w_i x_i$$

- $f(\text{net}) = \max(0, \text{net})$

- maxnet allows greater parallelism in execution as every computation is local to the node.

- maxnet & Hamming net are practically of not much use.

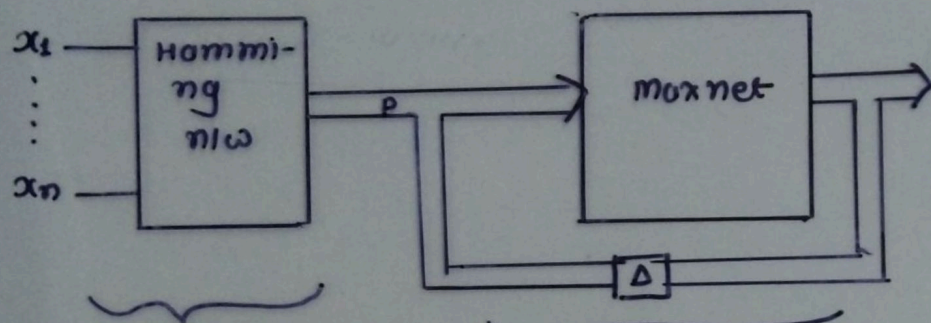
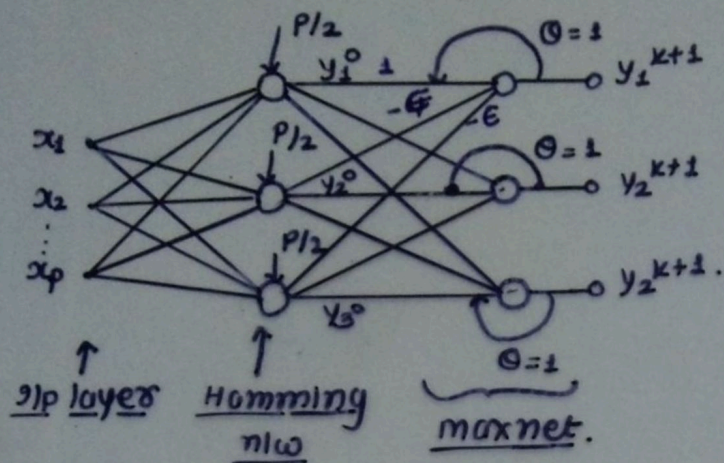
$\therefore$ combination of them is used.

Hamming net + maxnet = minimum HD classifier.

measures similarity + prevails node = winner node:
with highest
initial value

$$W = \begin{bmatrix} 1 & -1/5 & -1/5 & -1/5 & -1/5 \\ -1/5 & 1 & -1/5 & -1/5 & -1/5 \\ -1/5 & -1/5 & 1 & -1/5 & -1/5 \\ -1/5 & -1/5 & -1/5 & 1 & -1/5 \\ -1/5 & -1/5 & -1/5 & -1/5 & 1 \end{bmatrix}$$

- min HD classifier → if noisy or incomplete data given at o/p, it selects which stored vector is closer to o/p data. [i.e. selects winner node].



selects which stored vector is closer to unknown o/p vector

checks o/p's of Hamming n/w. o/p with max value prevails as winner & remaining all subside to zero.

Hamming distance = no of differing bits.

eg → y 1 -1 1 1 -1 1 1
x -1 -1 1 1 1 -1 1

Then →

& similarity betn x, y.

$$H(x, y) = \frac{1}{2} (n - \text{sum})$$

OR

$$-H(x, y) = \frac{1}{2} (\text{sum} - n)$$

$$\text{i.e. } H(x, y) = \frac{1}{2} [7 - 1] = \frac{6}{2}$$

$$\boxed{H(x, y) = 3}$$

$$S(x, y) = \frac{1}{2} (n + \text{sum})$$

$$= n - \frac{1}{2} (n - \text{sum})$$

$$= n - H(x, y)$$

$$= 7 - 3$$

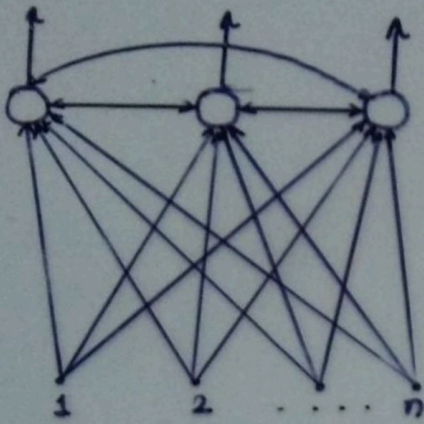
$$\boxed{S(x, y) = 4}$$

Hamming n/w → with threshold = $-\frac{n}{2}$ gives -ve of difference at o/p

Hamming n/w → with threshold = $+\frac{n}{2}$ " similarity betn two o/p's at o/p

• Simple Competitive Learning Algo →

(5)



- It consists of n nodes at ip & m nodes at olp.

- Each node in 1st layer is connected to each node in olp layer.

olp layer

- No. of nodes in olp layer are fixed only their connection wts vary.

- Appropriate wt update rule is used to determine connection wts from ip nodes ip layer to competitive layer.

- nodes in olp layer are connected to each other with inhibitory connections as of MAXNET.

- Train the n/w such that wt vector w_j associated with j th olp node represents vector of class of ip pattern.

- Initialize wts randomly.

- Training consist of two phases.

① Cometing phase → Randomly choose ip sample vector
- For this calculate olp of all nodes
- Determine winner node.

② Rewording phase → update wts of winner nodes.

Repeat procedure until all wts are stabilized.

• Algo →

1. Assume random wts.
2. Choose suitable learning const.
3. Present inp pattern i_k ($k=1 \dots n$)
4. compute distance bwn inp vector i_k & stored vector w_j ($j=1 \dots m$)
Euclidean Distance $\Rightarrow$

$$d_{kj} = \sqrt{\sum_{k=1}^n (i_k - w_j)^2}$$

5. Determine winner node w_j^* as $\leftarrow$ called as competitive learning
- as competition occurs to find the winner node among nodes of inp layer.
 $d_{kj^*} < d_{kj} \forall j$

6. update wts using,

$$\Delta w_j^* = \eta (i_k - w_j)$$

$$\& w_j(\text{new}) = w_j(\text{old}) + \eta [i_k - w_j(\text{old})] d_j$$

$$d_j = 1 \text{ for winner node}$$

$$d_j = 0 \text{ for other nodes.}$$

wt update rule is formulated such that wt vector associated with each node is as near as possible to all inp samples for which node is winner of competition.

7. Repeat procedure for all inp vectors.

$$8. \text{ centroid of cluster } j \quad w_j = \frac{1}{d_{j1}} \sum i_k d_{j1}$$

9. Repeat till n/w convergence.

Simple competitive algo updates wts to reduce quantization error defined by

$$E = \sum_p \|p - w(c(p))\|^2$$

• k-means clustering → Algo →

① Initialisation → choose k patterns from all d ip samples & treat them as cluster centroids.

- For each input pattern determine distance from every cluster

- Determine cluster closest to d ip pattern

- Assign input pattern to that vector.

- Repeat for All d ip patterns.

② Iterate — Recompute centroid for each cluster
- Repeat above procedure

③ Stop — when clusters are stable / min quantization error.

Simple competitive.

k-means.

Every time d ip sample is presented, winner node is updated by small amount.

After one complete iteration of d ip sample presentation, precise cluster centroid is calculated

If $k =$ no of data pts, quantization error is zero. useless.

Disadv →

- Quantization error has large no of local min.
∴ possibility of getting stucked into local min.
∴ quality of soln is far from optimal soln.

Adv →

1. Easy to understand, simple implementation.
2. Fast.

Observations →

(8)

1. If $n = 1$, convergence will never occur as node position will be completely changed to that of activating pattern.

previous eg - i_1 - presented $(1.1, 1.7, 1.8)$, w_3 is winner $= [1, 1, 1]$

$$\therefore w_3^1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + 1 \begin{bmatrix} 1.1-1 \\ 1.7-1 \\ 1.8-1 \end{bmatrix} = \begin{bmatrix} 1.1 \\ 1.7 \\ 1.8 \end{bmatrix}$$

If η is too high, convergence is not smooth.

If η is too low (0.01...), too many iterations will be req. for convergence.

2. Nlw is sensitive to choice of exact distance metric used.

Manhattan distance $= d(x, y) = \sum_1 |x_i - y_i|$

Hamming distance - can not be used, due to continuous values.

Euclidean distance $= \sqrt{\sum_{i=1}^n (x_i - w_j)^2}$

previous eg - when $i_2 = (0, 0, 0)$ was presented i_2 was winner with Hamming distance w_2 is the winner.

3. Initial wt values play some role in determining which node will be activated by which sample. **

eg → let $i_1 = (0.7, 0.7, 0.7)$ if patterns are presented in seq -
 $i_2 = (0.8, 0.8, 0.8)$ $i_1, i_2, i_3, i_4, i_1, i_2 - A \leftarrow i_3, B \leftarrow i_4, C \leftarrow i_1, i_2$
 $i_3 = (0, 0, 0)$ & if patterns are presented in seq →
 $i_4 = (0.9, 0.9, 0.9)$ $i_2, i_1, i_3, i_4, i_2, i_1 - A \leftarrow i_3, C \leftarrow i_2, i_4.$

4. Result of computation depends on which sequence of in which samples are presented to nlw, specially when n is not small. i.e no of clusters vary.

** if $w_0 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 5 \\ 0 & 5 & 0 \end{bmatrix}$ $A \leftarrow i_1, \dots, i_6$. all will converge to A.

if $w_0 = \begin{bmatrix} 0 & 0 & 0 \\ 1.01 & 1.01 & 1.01 \\ 1 & 1 & 1 \end{bmatrix}$ Two node positions are almost identical
 $A \leftarrow i_2, i_3, i_4, i_5$
 $B \leftarrow i_1$
 $C \leftarrow i_6$

5. Termination (criterion) $\rightarrow$

In the initial iterations diff nodes will be the winner in diff iterations
 In later iterations same node will be the winner in diff iterations for some
 sample - Terminate.

eg - present i_1 .

$$\left. \begin{array}{ll} \underline{I \Gamma I} \rightarrow w_3 & \underline{I \Gamma II} \rightarrow w_1 \\ \underline{I \Gamma II} \rightarrow w_2 & \underline{I \Gamma III} \rightarrow w_1 \\ \underline{I \Gamma III} \rightarrow w_1 & \underline{I \Gamma IV} \rightarrow w_1 \end{array} \right\} \text{ now is stabilized.}$$

- Also if ~~each~~ net change in each node position is zero at the end of one complete iteration, terminate.
 store each node position at beginning of previous iteration $\left[\begin{array}{l} \text{compute} \\ \text{re} \end{array} \right] \left[\begin{array}{l} \text{store each node position} \\ \text{at beginning of next it} \end{array} \right]$
 if $\downarrow$ no change terminate.

• observations →

(10)

1. If value of η too high, convergence is not smooth.

If value of $\eta = 1$, convergence will never occur as node position will be completely changed to that of activating pattern.

If $\eta = 0.01$, 100's of iterations will be required for convergence.

2. Nlw is sensitive to choice of exact distance metric used.

- Manhattan Distance → $d(x, y) = \sum_i |x_i - y_i|$

Substitute $\eta = 1$ in

- Euclidean Distance → $\sqrt{\sum_{i=1}^n (x_{ik} - w_{jk})^2}$

solved problem & see the result.

- Hamming Distance → No of differing bits ← (can not be used).

3. Initial values of wts play a important role in determining which node will be activated by which sample.

All connection wts should be assigned randomly bern 0 & 1.

4. Result of computations depends on sequence in which samples are presented to nlw, specially when η is not too small.
i.e no. of clusters vary.

5. Termination criterion →

a. If net change in each node positions is zero at the end of iterations. [i.e when there is no change in the wts in one complete iteration]. Use memory for storing same.

6. The memory can be used to reduce redundant calculations in later phase.

If node activated by each sample = node activated by sample in one iteration at previous iteration.

- It shows that nlw is stabilized & some nodes will continue to be activated in following iterations too.

1. No of o/p's are fixed →

if few several clusters will be combined into one class.

if many $\therefore$ it causes over classification.

$\therefore$ fixed.

• LVQ Algorithm —

- It's application of winners all take n/w.
- It shows how unsupervised learning mechanism can be used for supervised learning task.
- Each node is associated with arbitrarily chosen class label.
- If each cluster has roughly same no of patterns then for each class, no of nodes & no of training patterns that belongs to that particular task.
- learning rule for SCL is diff than LVQ.
- In LVQ, we determine winners node.
- if winner node belongs to some class as that of gip pattern, then bring wt vector closer to gip pattern
- All other nodes will be moved away from gip pattern.
- learning rate here is fun. of time $n(t)$

$$n(t) = \frac{1}{t} \quad \text{OR} \quad n(t) = a \left[1 - (t/A) \right] \quad \text{where } A, a = +ve \text{ const.} \\ \text{ \& } A > 1$$
- It increases speed of convergence to the state where wt vector will be stable & change a little with further gip sample presentation.

Algo $\rightarrow$ LVQ.

(12)

1. Initialize wts to random values.
2. choose suitable value of $\eta(t)$.
3. present ip pattern.
4. Determine winner node.
5. If winner node is of same class as ip pattern $w_j^* = w_j + \eta(t)(ix - w_j)$
" " " not " " " " " $w_j^* = w_j - \eta(t)(ix - w_j)$
6. Repeat step ② ④ ⑤ for all ip patterns.
7. Repeat complete procedure till convergence.

• Adaptive Resonance Theory → [ART] (13)

→ ART n/w is used for clustering.

→ ART is different as compare to other clustering Algo's with following points.

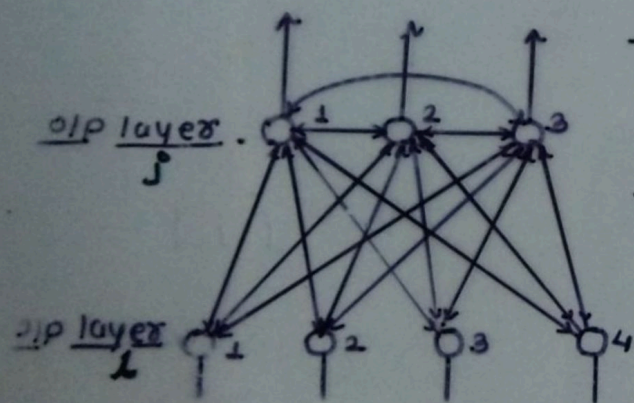
1. size of n/w [no of nodes in o/p layer] is not fixed, it dynamically changes depending on size of problem.
2. It allows user to control degree of similarity b/w members of same cluster by means of user defined const called as vigilance parameter ρ .

→ Adaptive Resonance

As n/w may generate new additional nodes to associate with given o/p pattern.

It is process of matching new o/p vector with one of cluster prototype stored in n/w.

→ ART N/w →



- It uses two layers of nodes with feed-forward & also feedback connections.

- no of nodes in o/p layer = size of pattern vector.

no of nodes in o/p layer = no of classes

- All connections are bidirectional.

- Every node in o/p layer is associated with certain prototype.

- let L = size of pattern vector
 J = no of nodes in o/p layer.

- then b_{j1} = bottom up weights
 t_{ij} = topdown weights.

- connection wts in both the direction may be different.
 i.e. $b_{j1} \neq t_{ij}$.

- ρ = degree of similarity b/w patterns of same class
 for 100% similarity $\rho = 1.0$
 Practically $\rho = 0.9$.

- It restricts all o/p's to be binary valued. [0 or 1]

• Learning Process —•

1. Present o/p pattern.
2. o/p layer receives & stores o/p pattern
3. o/p node responds only to certain prototypes.
4. NW tries to find match betn o/p pattern & prototype vector.
5. As a measure of matching similarity betn them is calculated.
6. Similarity is compared with initially assumed value of ϵ .
7. If similarity $> \epsilon$, match is accepted
8. Update wts of corresponding node in o/p layer $[b_j, t_{ij}]$
9. If similarity $< \epsilon$, corresponding node is rejected.
NW tries next o/p node in sequence. Process continues till we find suitable match with o/p pattern.
10. If no node in o/p layer responds with suitable prototype, NW generates additional node to associate with o/p pattern.

• ALGO $\rightarrow$ ART.

(15)

1. Initialize ϵ , $t_{ij} = 1$ & $b_{ji} = \frac{1}{1+n}$ where n = size of pattern vector.
2. Present 1st input pattern x_1 .
3. Compute $y_j = \sum_{i=1}^n b_{ji} \cdot x_i$.
4. Detect $y_j^* > y_j \forall j (1 \dots n)$ i.e. determine winner node.
5. Calculate similarity as $\text{similarity} = \frac{\sum x_i t_{ij}}{\sum x_i}$
6. If $\text{similarity} > \epsilon$ update wts as
$$b_{ji}^* = \frac{t_{ij}(0) \cdot x_i}{0.5 + \sum t_{ij}(0) \cdot x_i} \quad \& \quad t_{ij}^* = t_{ij}(0) \cdot x_i$$
7. go to step ②
8. If $\text{similarity} < \epsilon$, select the next winner node.
9. Repeat steps from ⑤ to ⑧ until desired match is selected.
10. If no match is found, generate additional node with
$$t_{ij} = x_i \quad \& \quad b_{ji} = \frac{x_i}{0.5 + \sum_{i=1}^n x_i t_{ij}}$$
11. Repeat steps ② to ⑩ for all patterns in training set.
12. cycle now the one or more sequences till it stabilizes.

• Distance based learning →

If wt vectors of many nodes are in vicinity of an gip pattern

→ competitive learning → only winner node is updated.

som → winner & neighbourhood nodes are updated.

Distance based learning → All the nodes other than winner node are updated.

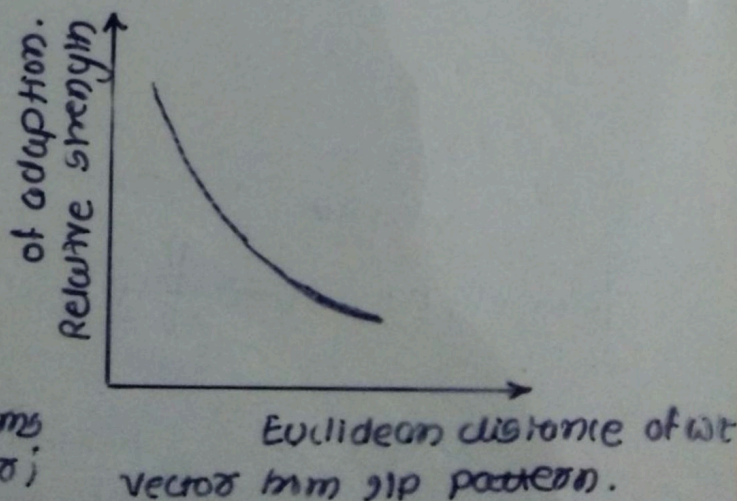
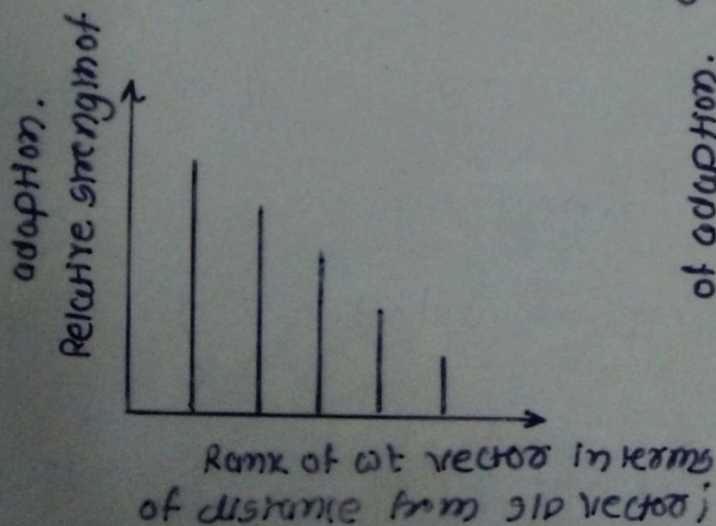
Two Algo's →

1. max Entropy →

- All the wt vectors get affected due to presentation of gip pattern.
- Change in wt vector is nonlinear function of distance betⁿ gip vector & wt vector.
- All wt vectors are drawn closer to gip vector.
- Wt vector far away from gip vector is modified by small amount.
- Wt vector closer to gip vector is modified by large amount.
- magnitude of wt change also depends upon temp.

$$\Delta w_{ij} = n(i - w_{ij}) \left[\frac{e^{\left[\frac{-|i - w_{ij}|^2}{T} \right]}}{\sum_k \exp \left[\frac{-|i - w_{kj}|^2}{T} \right]} \right]$$

- If $T \downarrow$ steadily, wt vector closer to gip vector will have significant updates in algorithm.



• NEURAL GAS learning.

• maximum Entropy.

• NEURAL GAS →

- Due to presentation of gip vectors, all wt vectors are affected.
- Amount of wt modification depends on distance betⁿ wt vectors & gip vectors.
- Nodes are ranked by their proximity to gip vectors
- magnitude of wt change depends on rank of the node.

- If $i = \text{input vector}$ & $w = \text{wt vector}$ then

$$k_j(i, w) = \text{rank}$$

= no of wt vectors whose distance from i is smaller than the distance from i to w_j

- For j^{th} node calculate distance from pattern i .

For all other nodes repeat same.

- Compare all other node distances with winner node & find out its rank.

$$\Delta w_j = \eta (i - w_j) h(k_j(i, w))$$

- $\eta = \{0, 1\}$, $h = \text{monotonically decreasing function}$ with $h(0) = 1$.

$$\& h(x) = e^{-x/\lambda}$$

- normally $\lambda > 0$ is decay const.

if $\lambda = 0$, wt update is same as competitive learning.

• It is usually slower than competitive algo due to calculation of rank for every gip pattern.